

Step 2 - Transformations on Banking Data

Phase A · Filters · Flags · Joins · Orphans · ~20 min

Goal: Master daily-driver transformations on your data: filter, when/otherwise flags, joins (inner/left/anti), orphan detection, branch leaderboard. Artifact: 02_transformations.ipynb

LEARN

- 1. Transformations chain and stay lazy.** Each returns a new DataFrame; nothing executes until an action (show/count/write) - Spark optimizes the whole plan.
- 2. when/otherwise is CASE WHEN.** Every banking flag is born this way: FINTRAC tiers, risk bands, segments.
- 3. Joins turn rows into stories.** A transaction alone says '\$15,000 e-Transfer'; joined to customer and branch it says who, where, and how new the account is.
- 4. The anti-join is your data-quality detective** - rows on the left with NO match on the right = orphan transactions. Banks run this nightly.

LAB — notebook 02_transformations (Serverless)

Cell 1 — Reload (each notebook stands alone): load df_txn with the explicit txn_schema from Step 1, plus df_customer and df_branch with header=True.

Cell 2 — Filter, the FINTRAC slice:

```
df_large = df_txn.filter(F.col("amount_cad") >= 10000)

# Multiple conditions: & not 'and', parentheses MANDATORY
df_large_ettransfer = df_txn.filter(
    (F.col("transaction_type") == "INTERAC_ETRANSFER") &
    (F.col("amount_cad") >= 10000)
)
print(df_large.count(), df_large_ettransfer.count())
```

Cell 3 — Flags with when/otherwise:

```
df_flagged = df_txn \
    .withColumn("hst_13pct", F.round(F.col("amount_cad") * 0.13, 2)) \
    .withColumn("fintrac_flag",
        F.when(F.col("amount_cad") >= 10000, "LCTR_REPORTABLE")
        .when(F.col("amount_cad") >= 9000, "STRUCTURING_RISK")
        .otherwise("NORMAL")) \
    .withColumn("value_band",
        F.when(F.col("amount_cad") < 100, "MICRO")
        .when(F.col("amount_cad") < 1000, "SMALL")
        .when(F.col("amount_cad") < 10000, "MEDIUM")
        .otherwise("LARGE"))

df_flagged.groupBy("fintrac_flag").count().show()
```

The \$9,000-\$9,999 STRUCTURING_RISK band is a real AML pattern: deliberately staying just under the reporting threshold. Naming it in interviews signals genuine domain knowledge.

Cell 4 — Joins, enrich the transaction:

```
df_enriched = df_flagged.join(
    df_customer.select("customer_id", "first_name", "last_name", "city", "province"),
    on="customer_id", how="inner"
).join(
    df_branch.select("branch_id", "branch_name"),
    on="branch_id", how="left")

df_enriched.filter(F.col("fintrac_flag") == "LCTR_REPORTABLE") \
    .select("transaction_id", "first_name", "last_name", "city",
           "amount_cad", "transaction_type", "branch_name") \
    .orderBy(F.col("amount_cad").desc()).show(10, truncate=False)
```

Cell 5 — Anti-join, the orphan hunt:

```
df_orphans = df_txn.join(df_customer, on="customer_id", how="left_anti")
print(f"Orphan transactions: {df_orphans.count()}") # expect 0 here
```

Cell 6 — Branch leaderboard (the phone-screen query):

```
df_enriched \
    .filter(F.col("transaction_type") == "INTERAC_ETRANSFER") \
    .filter(F.col("province") == "ON") \
    .groupBy("branch_id", "branch_name") \
    .agg(F.count("*").alias("txn_count"),
         F.round(F.sum("amount_cad"), 2).alias("total_cad"),
         F.countDistinct("customer_id").alias("unique_customers")) \
    .orderBy(F.col("total_cad").desc()).show(5, truncate=False)
```

COMMIT + CHECKPOINT

Export to `pyspark/02_transformations.ipynb`. Pass criteria: flag counts roughly 300 LCTR / 120 structuring-risk, 0 orphans, sensible Ontario leaderboard. If you can write Cell 6 from memory, you pass most PySpark phone screens.